

Computer Applications in Modern Information Societies (2)

Part 1: Programming Fundamentals

Lecture 1 &2

2025

Introduction: Types of Problems

- Problems with *Algorithmic Solutions*
 - Problems that can be solved with a **series of actions**
 - Examples: baking a cake, alphabetizing 10,000 names
- Problems with *Heuristic Solutions*
 - Problem solution **requires reasoning** based on knowledge and experience, and a process of trial and error
 - Solutions cannot be reached through a direct set of steps
 - Examples: how to buy the best stock or expand the company?
 - Difficult to evaluate the solution

Introduction: Algorithmic problems

- In computing, we focus on the type of problems categorically known as *algorithmic problems*,
- In *algorithmic problems*, solutions are expressed in the form of a *well-defined procedure (algorithm)* consisting of a *set of steps (instructions)*, that takes some value or set of values, as *input*, do some *processing*, and produces some value or set of values, as *output*.



Input

Input can be a
value or some
set of values



```
def function (Input, Parameters):  
    Code .....  
    Param = 0, 10, 100  
    Code .....  
    Param = 1, 10, 100  
    Code .....  
    if Param == 0: return  
    Code .....  
    Code .....
```

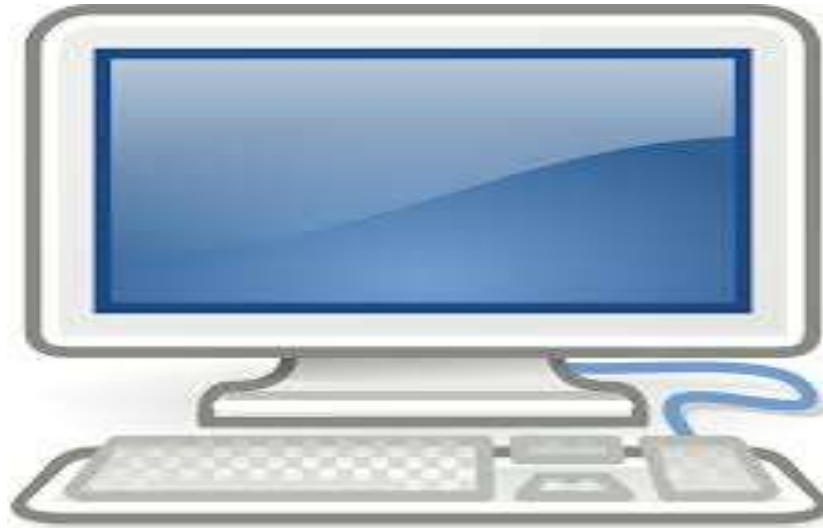
Algorithm

Precisely defined
procedure written in
programming language



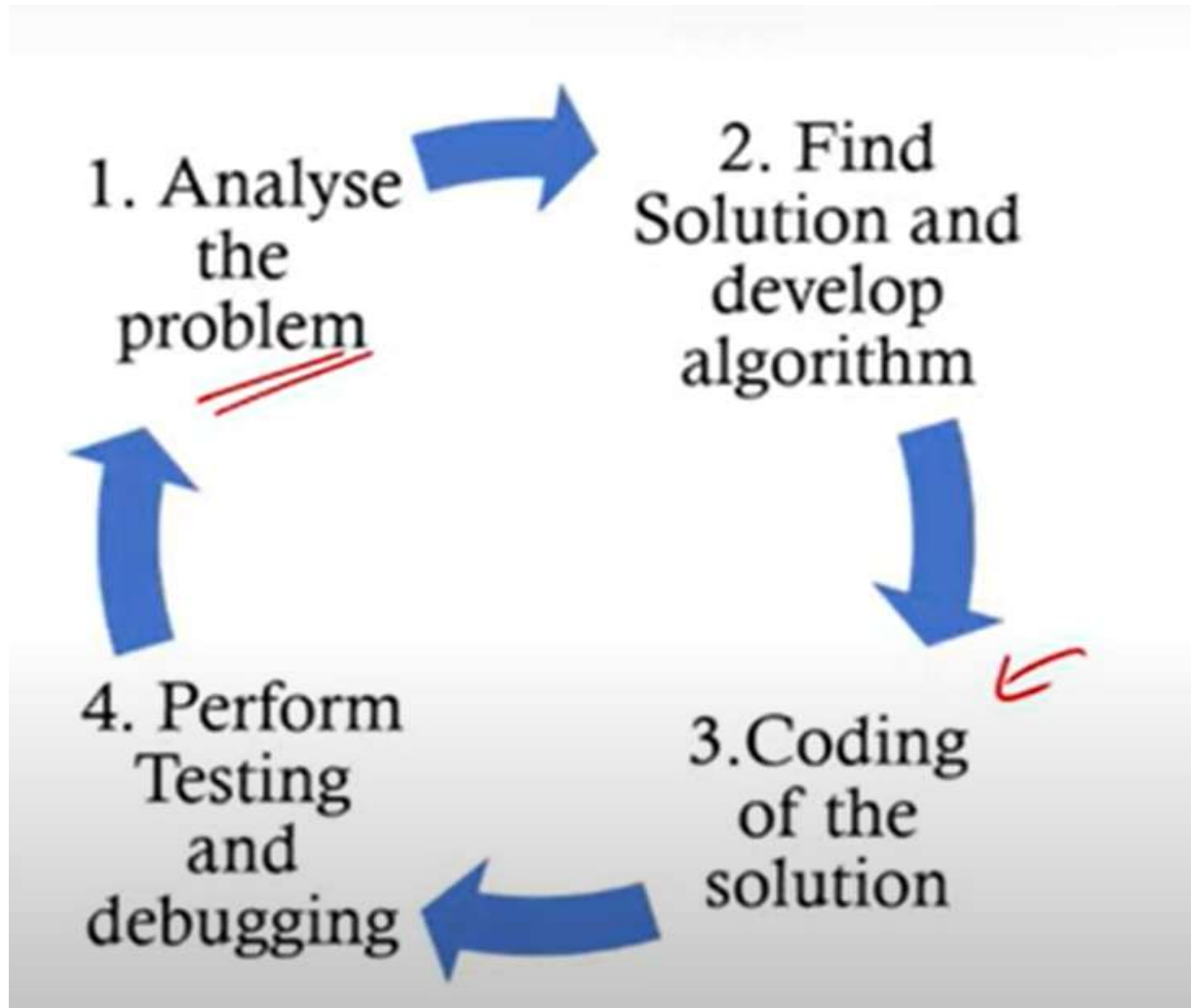
Output

Output can be
some values or
set of values.



- Has **no intelligence** or thinking power.
- The **Intelligence Quotient (I.Q)** of a computer is **zero**.
- A computer performs many tasks exactly in the same manner as it is told to do.
- People are better than computers at developing **heuristic solutions**.
- **Computers are built to deal with algorithmic solutions, which are often difficult or very time consuming for humans.**
- The field of computers that deals with heuristic types of problems is called **artificial intelligence**.

Programme Development Life Cycle



1. Analyzing the problem

- Understanding the problem
- **Inputs** to the problem
- **Outputs** expected from the problem
- Special constraints or **conditions**
- **Formulas** or equations to be used

2. Developing the algorithm

An algorithm is a plan, a logical step-by-step process for solving a problem.

- Think of all possible solutions
- Follow modular approach- breaking down into simple steps
- Algorithms are normally written as a **flowchart** or in **Pseudo-code**

3. Coding

It's a technical term used for writing a program

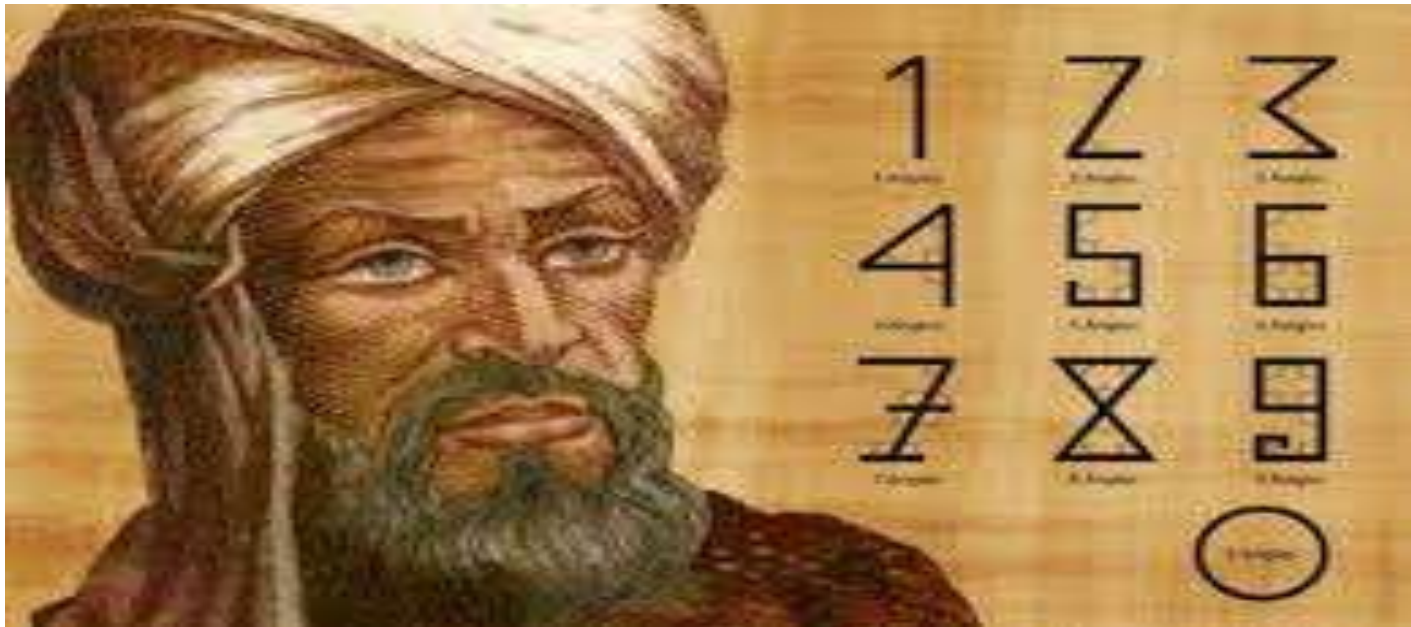
- It is the process of transforming design (Algorithm, Pseudocode, Flowchart) in a computer program using any Programming language.
- Think about efficiency of solution in terms of speed, performance and effectiveness

4. Testing and debugging

- **Testing** is the process of finding errors in the program
- **Debugging** is the process of correcting errors in the program
- It involves **finding errors and correcting them**

Algorithms

- The word *algorithm* comes from the name of a mathematician Abu Ja'far Mohammed ibn-i Musa al Khowarizmi.



Algorithms: Real Life Examples

■ Many Real-Life Examples

- **Cooking:** Recipe for preparing a dish
- **Origami:** The Art of Paper Folding
- **Directions:** How to go to Cairo Airport

□ **Example:** how do you make a cup of tea?

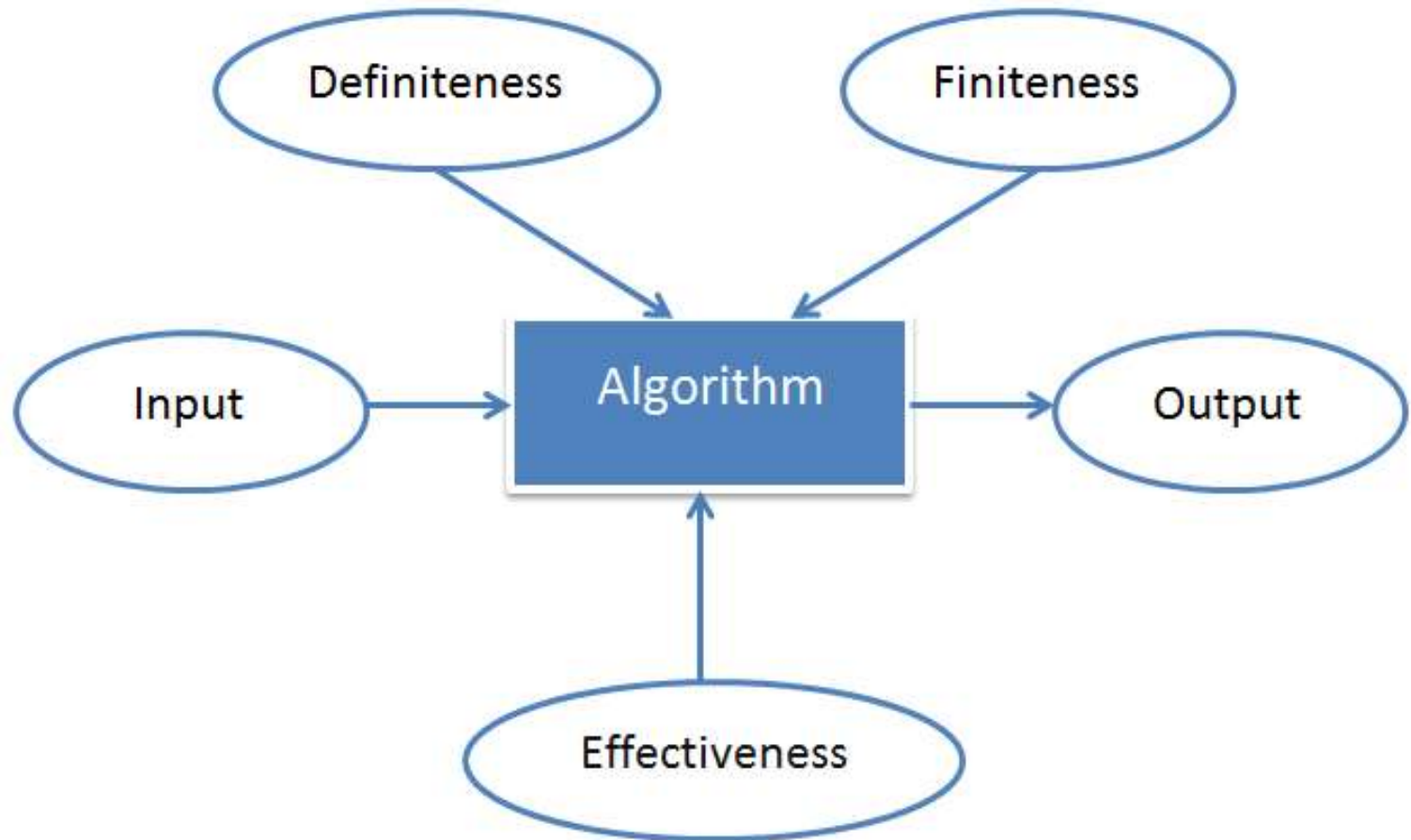


1. Boil kettle
2. Put tea in cup
3. Pour boiling water into cup

Algorithms

- It may be possible to solve a problem in *more than one way*, resulting in *more than one algorithm*.
- The choice of various algorithms depends on some factors such as *reliability*, *accuracy* and *easy to understand and modify*.
- An important factor in the choice of algorithm is the *execution time*, after writing code in a high-level language. The algorithm needs the *least time* when executed is considered the best (*example: sorting algorithms*).

Properties of an efficient algorithm



Properties of an efficient algorithm

- **Finiteness:** An algorithm must terminate in a finite number of steps
- **Definiteness:** Each step of the algorithm must be clear and unambiguous.
- **Effectiveness:** Each step must be effective, in the sense that it should be primitive easily convertible into program statement , can be performed exactly in a finite amount of time.(steps must be doable).
- **Input/output:** Each algorithm must take zero, one or more quantities as input data and produce one or more output values.
- **Generality:** The algorithm must be complete in itself so that it can be used to solve problems of a specific type for any input data.



Some algorithms are harder than others

- **Some algorithms are easy**

- ☐ Finding the largest (or smallest) value in a list
- ☐ Finding a specific value in a list

- **Some algorithms are a bit harder**

- ☐ Sorting a list

- **Some algorithms are very hard**

- ☐ Finding the shortest path between two cities

- **Some algorithms are essentially impossible**

- ☐ Factoring large composite numbers

Steps involved in algorithm development

- Step1: *Identification of input*: For an algorithm, there are quantities to be supplied called *input* and these are *fed externally*. The input is to be identified first for any specified problem.
- Step2: *Identification the processing operations*: All the calculations to be performed in order to lead to output from the input are to be identified in an *orderly manner*.
- Step3: *Identification of output*: From an algorithm, at least one quantity is produced, called “*Output*”, for any specified problem.

Three constructs

```
do action 1  
do action 2  
...  
...  
do action  $n$ 
```

a. Sequence

```
if a condition is true,  
then
```

```
do a series of actions
```

```
else
```

```
do another series of actions
```

b. Decision

```
while a condition is true,
```

```
do action 1  
do action 2  
...  
...  
do action  $n$ 
```

c. Repetition

Example 1

Find the average of three numbers

- **Step 1:** Read the numbers a, b, c
- **Step 2:** Compute the sum of a, b and c
- **Step 3:** Divide the sum by 3
- **Step 4:** Store the result in variable d
- **Step 5:** Print the value of d
- **Step 6:** End

Example 2

Write an algorithm to calculate the simple interest using the formula. $\text{Simple interest} = P * N * R / 100$

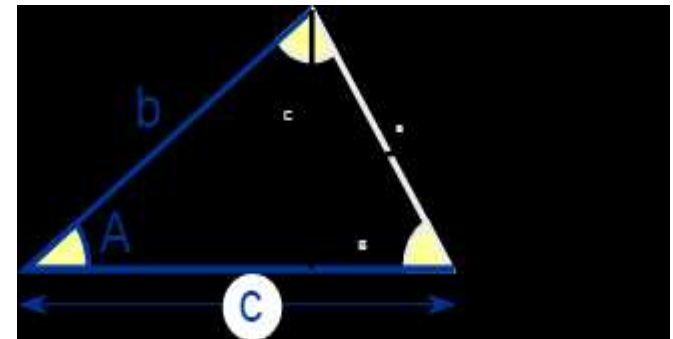
Where P is principle Amount, N is the number of years and R is the rate of interest.

- **Step 1:** Read the three input quantities' P, N and R.
- **Step 2:** Calculate simple interest as $\text{Simple interest} = P * N * R / 100$
- **Step 3:** Print simple interest.
- **Step 4:** End

Example 3

Write an algorithm to find the area of the triangle. Let b , c be the sides of the triangle ABC and A the included angle between the given sides.

- **Step 1:** Input the given elements of the triangle namely sides b , c and angle between the sides A .
- **Step 2:** $\text{Area} = (1/2) * b * c * \sin A$
- **Step 3:** Output the Area
- **Step 4:** End



Example 4

Write an algorithm to calculate the perimeter and area of rectangle. Given its length and width.

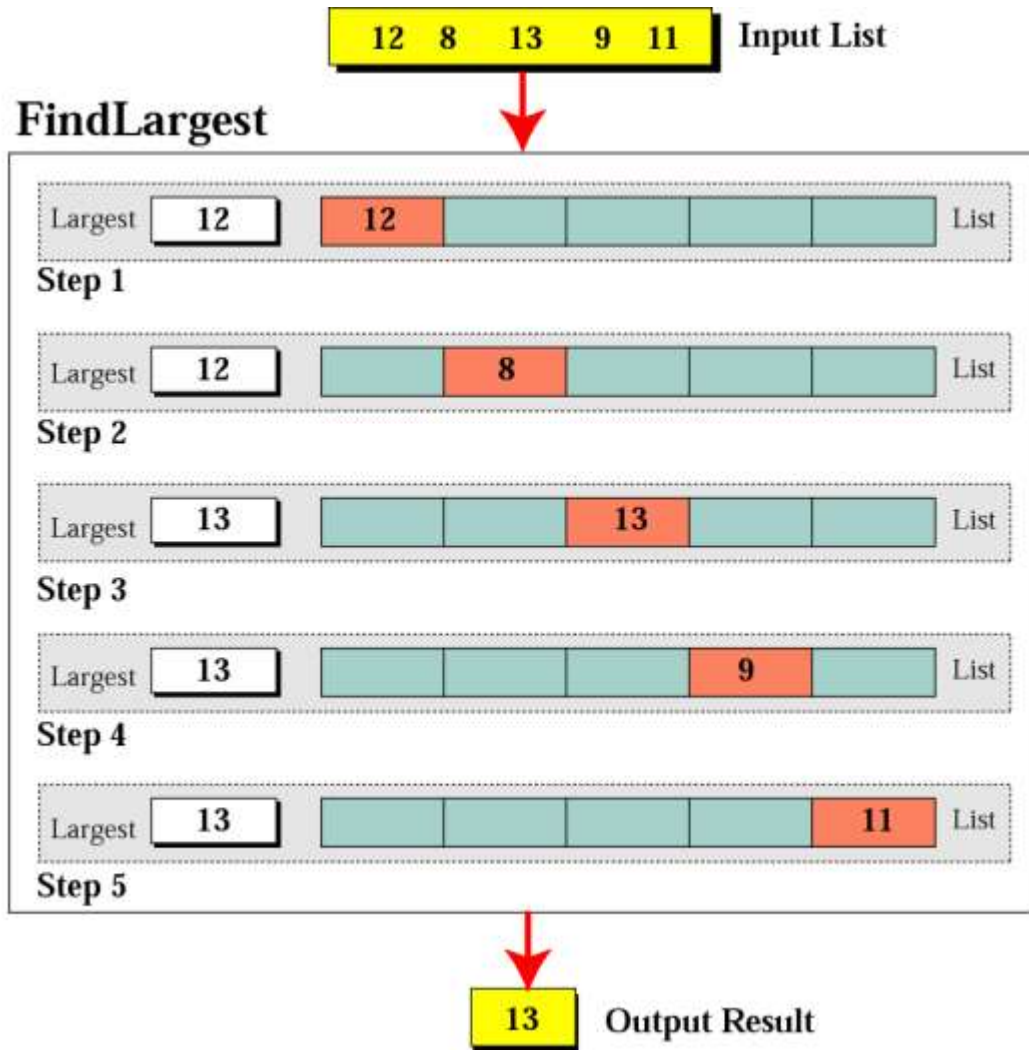
- **Step 1:** Read length of the rectangle.
- **Step 2:** Read width of the rectangle.
- **Step 3:** Calculate perimeter of the rectangle using the formula $\text{perimeter} = 2 * (\text{length} + \text{width})$
- **Step 4:** Calculate area of the rectangle using the formula $\text{area} = \text{length} * \text{width}$.
- **Step 5:** Print perimeter.
- **Step 6:** Print area.
- **Step 7:** End

Example 5

Write an algorithm to find the largest of three numbers X, Y, Z.

- **Step 1:** Read the numbers X, Y, Z.
- **Step 2:** if ($X > Y$)
- Big = X
- else Big = Y
- **Step 3:** if ($\text{Big} < Z$)
- Big = Z
- **Step 4:** Print the largest number i.e. Big
- **Step 5:** Stop.

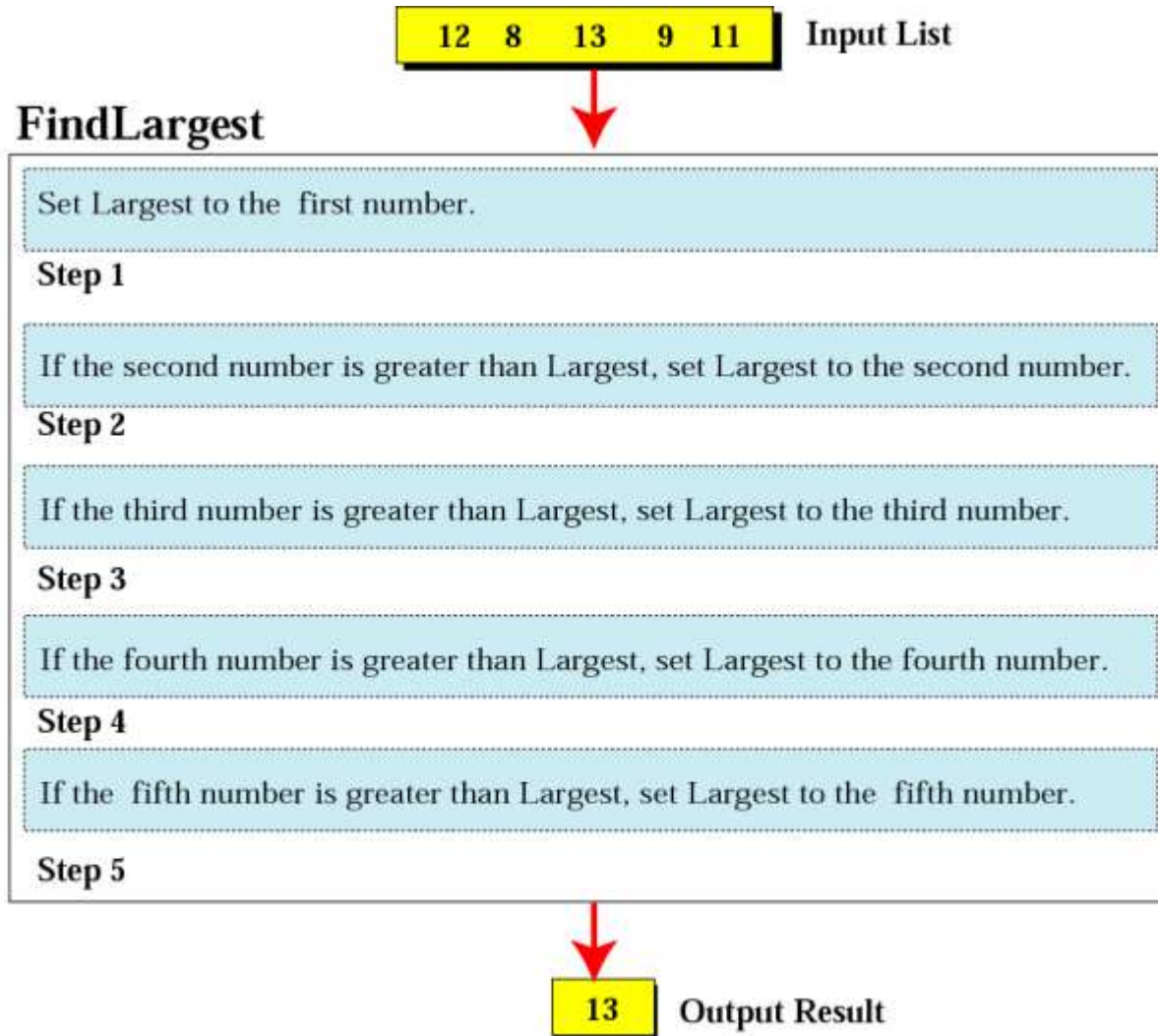
Finding the largest integer among five integers



12 8 13 9 11

- **Step 1:** Set the 12 to the Largest numbers.
- **Step 2:** 12 is larger than 8 ,there is no need to change the value of Largest.
- **Step 3:** 13 is larger than 12 , the algorithm changes the value of Largest to 13 and moves to the next step.
- **Step 4:** Nothing is changed in this step, because 13 is larger than 9.
- **Step 5:** Again nothing is changed , because 13 is larger than 11.
- **Step 6:** print largest

Defining actions in FindLargest algorithm



FindLargest refined

12 8 13 9 11

Input List



FindLargest

Set Largest to 0.

Step 0

If the current number is greater than Largest, set Largest to the current number.

Step 1



If the current number is greater than Largest, set Largest to the current number.

Step 5

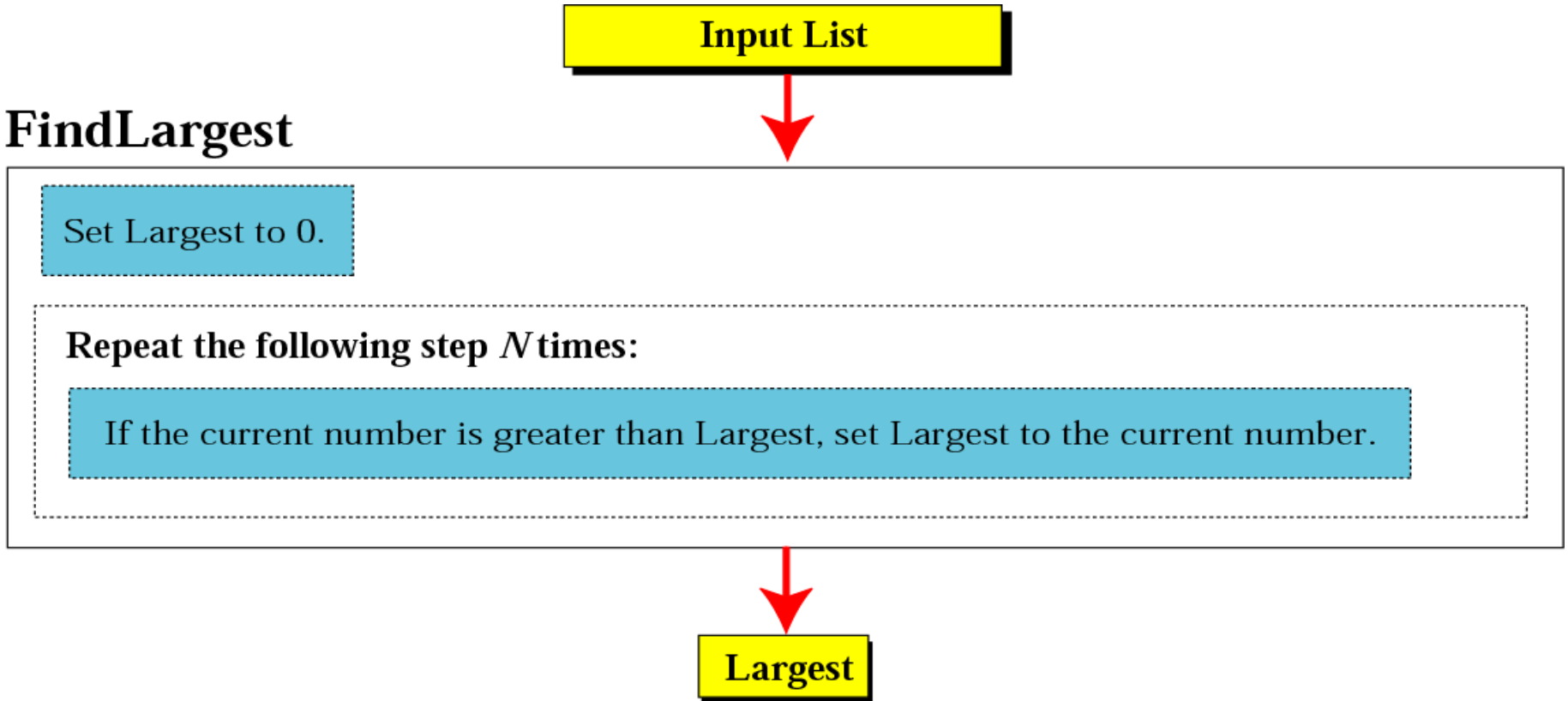


13

Output Result

Generalization of FindLargest

FindLargest



<https://youtu.be/jn9lZVGKWhg>

Three constructs

```
do action 1  
do action 2  
...  
...  
do action  $n$ 
```

a. Sequence

```
if a condition is true,  
then
```

```
do a series of actions
```

```
else
```

```
do another series of actions
```

b. Decision

```
while a condition is true,
```

```
do action 1  
do action 2
```

```
...
```

```
...
```

```
do action  $n$ 
```

c. Repetition

Loop Design

...

I = 1

← *Initializing a Control Variable*

While (I <= 100) Do

← *Testing Loop Condition*

Print “Hello Feps”

← *Statement (s) to repeat*

I = I + 1

← *Updating the Control Variable*

...

This algorithm prints the message “Hello Feps” 100 times

Loop Design

...

Counter = 1



Initializing a Control Variable

While (Counter <= 10) Do



Testing Loop Condition

Print Counter



Statement (s) to repeat

Counter = Counter + 1



Updating the Control Variable

...

This algorithm prints the numbers between 1 and 10 to the screen

Example 6

Write an algorithm to find the factorial of a given number N ($N! = 1 * 2 * 3 \dots * N$)

Step1: Read N

Step2: $PROD = 1$

Step3: $I = 2$

Step4: **While** ($I \leq N$) **do**

Step5: **PROD = PROD * I**

Step6: **I = I + 1**



Step7: Write “Factorial of ”, N , “ is ”, $PROD$

Step8: End

Tracing: Testing the algorithm

Will the algorithm work to get factorial of 4? ($4! = 1*2*3*4$)

Step1: Read N

Step2: PROD = 1

Step3: I = 2

Step4: **While (I <= N) do**

Step5: **PROD = PROD * I**

Step6: **I = I + 1**

Step7: Write “Factorial of ”, N, “ is ”, PROD

Step8: End

N = 4

PROD = 1

I = 2

IF I (2) <= N (4) ? Yes

PROD = 1 * 2 = 2

I = 2 + 1 = 3

IF I (3) <= N (4) ? Yes

PROD = 2 * 3 = 6

I = 3 + 1 = 4

IF I (4) <= N (4) ? Yes

PROD = 6 * 4 = 24

I = 4 + 1 = 5

IF I (5) <= N (4) ? NO

Factorial of 4 is 24

Example 6

Write an algorithm to find the factorial of a given number N

- Step 1: $PROD = 1$
- Step 2: $I = 0$
- Step 3: read N
- Step 4: While $I < N$ do
 - $I = I + 1$
 - $PROD = PROD * I$
- Step 5: Write “Factorial of”, N, “is”, PROD
- Step 6: End

Example 7

Write an algorithm to find the sum of given data values until a negative value is entered.

NEW_VALUE: 5, 10, 7, ..., -1

Step1: SUM = 0

Step2: Read NEW_VALUE

Step3: **While (NEW_VALUE >= 0) Do**

Step4: SUM = SUM + NEW_VALUE

Step5: Read NEW_VALUE



Step6: Write “The sum is ” SUM

Step7: End

Tracing: Testing the algorithm

Will the algorithm work to get the sum of these three numbers 5, 10, 7?

Step1: SUM = 0

Step2: Read NEW_VALUE

Step3: While (NEW_VALUE >= 0) Do

Step4: SUM = SUM + NEW_VALUE

Step5: Read NEW_VALUE

Step6: Write “The sum is ” SUM

Step7: End

SUM = 0

NEW_VALUE = 5

IS NEW_VALUE (5) >= 0 ? YES

SUM = 0 + 5 = 5

NEW_VALUE = 10

IS NEW_VALUE (10) >= 0 ? YES

SUM = 5 + 10 = 15

NEW_VALUE = 7

IS NEW_VALUE (7) >= 0 ? YES

SUM = 15 + 7 = 22

NEW_VALUE = -1

IS NEW_VALUE (-1) >= 0 ? NO

The sum is 22